Perceptron Learning Algorithm

AI — Machine Learning Foundations · Session 3, Practical Exercise

Mario Sánchez García

September 9, 2026

Files

- `perceptron.py` — full implementation (parts a–e). Run with `python3 perceptron.py`.
- `b_perceptron_N100.png` — plot for part (b).
- `d_updates_hist.png` — update-count histograms, $N = 100$ vs. $N = 1000$.
- `resultados.txt` — raw console output of the run.

Setup

Target function f. A random line through two random points of $[-1, 1]^2$, represented as a weight vector $\mathbf{w}_f = [b, w_1, w_2]$ with

$$f(\mathbf{x}) = \operatorname{sign}\left(\mathbf{w}_f^\top [1, x_1, x_2]\right).$$

Dataset D. N points $\mathbf{x}_n$ drawn uniformly from $[-1, 1]^2$, labeled $y_n = f(\mathbf{x}_n)$. By construction the set is **linearly separable**, so the PLA is guaranteed to converge (perceptron convergence theorem).

PLA. Starting from $\mathbf{w}(0) = \mathbf{0}$, at each iteration a **randomly chosen** misclassified point $(\mathbf{x}(t), y(t))$ is picked and the weights are updated as

$$\mathbf{w}(t+1) = \mathbf{w}(t) + y(t)\,\mathbf{x}(t)$$

until no misclassified point remains.

Quality of g. $P[f(\mathbf{x}) \neq g(\mathbf{x})]$ is estimated by Monte Carlo sampling on fresh points (20,000 points for the repeated experiments, 100,000 for the single plotted run). This quantity is the perceptron's E_{out}.

Every experiment is repeated **100 times** (20 for part e) and both mean and median are reported — a single run has enormous variance and is not informative on its own.

(a) + (c) — $N = 100$

	mean	median	min	max
Updates until convergence	**109.5**	68	5	533
$P[f \neq g]$	**0.0124**	0.0100		

The algorithm always converges, and fairly quickly: in half the runs it takes fewer than about 70 updates. The final hypothesis g agrees with f over roughly **98.8%** of the plane.

The distribution of update counts is strongly right-skewed (mean 109 $\gg$ median 68): convergence is fast almost every time, but occasionally a dataset happens to place points very close to the target line and the algorithm needs many more corrections. This matches the convergence bound $t \leq R^2\|\mathbf{w}_f\|^2/\rho^2$, which depends on the **margin** ρ — the distance from the closest point to the separating line: a small margin means many iterations.

(b) — Plot

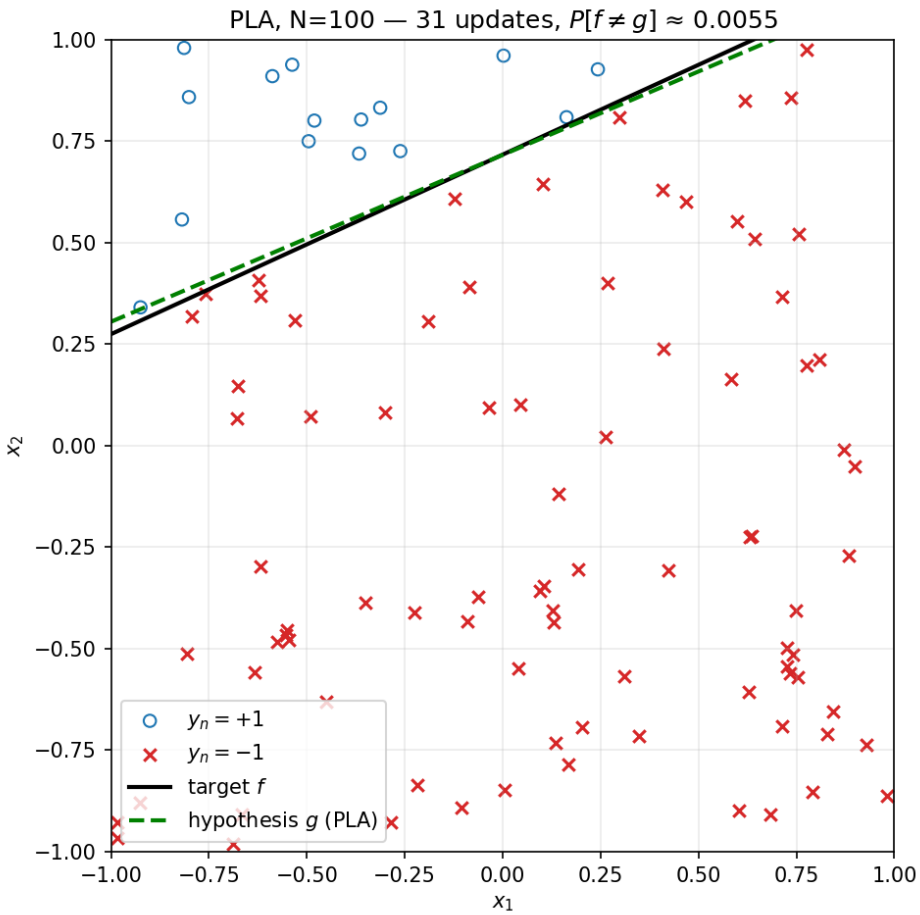

Figure 1: Single PLA run, $N = 100$: 31 updates, $P[f \neq g] \approx 0.0055$.

Points with $y_n = +1$ are shown as blue circles and $y_n = -1$ as red crosses; the solid black line is the target f, and the dashed green line is the PLA's final hypothesis g. In this particular run: 31

updates and $P[f \neq g] \approx 0.0055$.

The figure shows that g separates all 100 points perfectly ($E_{in} = 0$) but does **not** coincide exactly with f: the PLA stops as soon as there are no errors on the sample — it does not search for the maximum margin (that would require an SVM). The gap between the two lines is precisely the E_{out}.

(d) — $N = 1000$, comparison with (a)

	$N = 100$	$N = 1000$	change
Updates (mean)	109.5	**1325.4**	$\times 12.1$
Updates (median)	68	559	$\times 8.2$
$P[f \neq g]$ (mean)	0.0124	**0.0011**	$\div 11.0$

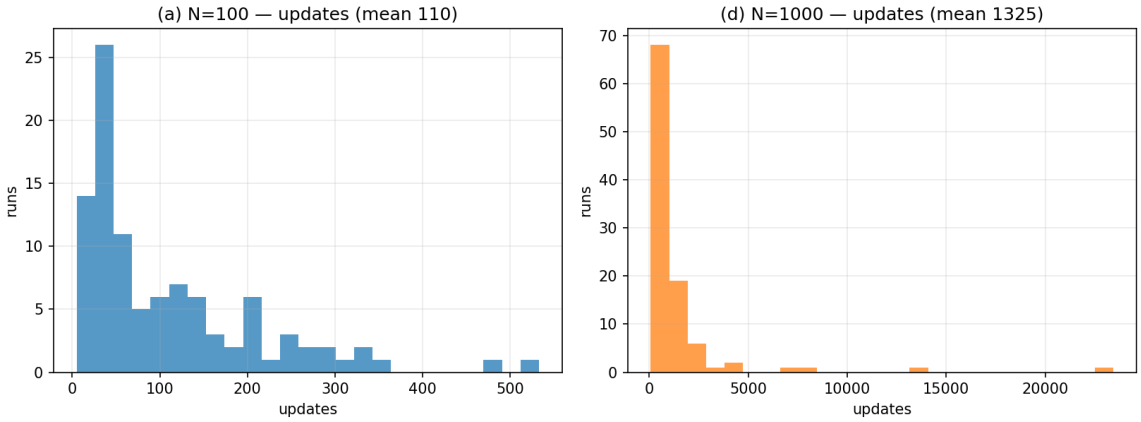

Figure 2: Distribution of update counts across 100 runs, $N = 100$ vs. $N = 1000$.

Two opposite effects appear, both expected:

1. **Convergence takes much longer** (roughly $\times 12$). With more points, it becomes more likely that some point falls very close to the target line, shrinking the margin ρ and pushing up the iteration bound.
2. **Generalization improves substantially** (roughly $\times 11$ lower error). More data constrains the set of lines compatible with the sample more tightly, so g stays much closer to f. This is the classic $E_{out} \sim d_{VC}/N$ relationship: for a 2-D perceptron ($d_{VC} = 3$), the out-of-sample error falls off approximately as $1/N$.

Conclusion: more data means more computational cost, but a better hypothesis.

(e) — $d = 10$, $N = 1000$

Generalizing to higher dimension: now $\mathbf{x}_n \in \mathbb{R}^{10}$ and $\mathbf{w} \in \mathbb{R}^{11}$ (including the bias term). To guarantee linear separability, labels are generated from a random $\mathbf{w}_f$:

$$y_n = \operatorname{sign}\left(\mathbf{w}_f^\top [1, \mathbf{x}_n]\right).$$

The code is **unchanged** — the PLA does not depend on the dimensionality.

	mean	median	min	max
Updates until convergence	**5176.2**	4355.5	1551	13889

At the same $N = 1000$, going from 2 to 10 dimensions multiplies the number of updates by roughly 4. Higher dimensionality means many more degrees of freedom to fit (11 weights instead of 3), and the relative margin of the points shrinks, so the algorithm needs far more corrections. It still converges every time, since the data is separable by construction.

Summary

- The PLA always converges when the data is linearly separable, and the number of updates depends strongly on the **margin**, not just on N.
- The number of updates **increases** with both N and the dimension d.
- $E_{out} = P[f \neq g]$ **decreases** with N (roughly as $1/N$).
- The PLA guarantees $E_{in} = 0$, but it does not maximize the margin: g is never exactly equal to f.